Informe técnico — AdelitaGOAT

Proyecto escolar: aplicación móvil para apartar comida en puestos cercanos a la Universidad Politécnica de Chiapas

Versión del informe: 10 de noviembre de 2026

Estado: aplicación y API desarrolladas; backend público y base de datos desplegados; APK de demostración compilado

1. Resumen del producto

AdelitaGOAT permite que un estudiante consulte el menú de un puesto cercano a la universidad, seleccione productos, confirme un apartado y reciba un folio. El vendedor consulta las comandas, las prepara y marca su entrega. El pago del MVP se realiza **en efectivo al recoger**: la aplicación no procesa tarjetas, transferencias ni dinero digital.

La idea responde a una situación concreta: los recesos entre clases son cortos y la fila de un puesto puede consumir parte importante de ese tiempo. La aplicación adelanta la elección y la comunicación del pedido; la preparación y entrega siguen a cargo del comercio.

2. Usuarios y recorridos principales

Usuario	Recorrido
Cliente	Se registra, inicia sesión, explora puestos y productos, busca y filtra, añade productos de un mismo puesto al carrito, confirma el apartado, consulta folio y estado, recoge y paga en efectivo. Puede guardar favoritos y calificar un pedido entregado.
Vendedor	Se registra, crea o edita su puesto con nombre, descripción, ubicación concreta, categoría, tiempo estimado y estado abierto/cerrado; carga productos y precios; cambia disponibilidad; consulta pedidos y avanza sus estados hasta la entrega.

Las cuentas usan correo y contraseña. La API conserva las sesiones y verifica el rol y la propiedad del puesto antes de permitir cambios. Cuando todavía no existe un puesto, la app muestra un estado vacío y ofrece al vendedor el formulario para registrarlo.

3. Alcance y reglas de negocio

- Un carrito y un pedido contienen productos de **un solo puesto**.
- El servidor consulta los precios vigentes y calcula el total; el cliente no determina el importe confiable.
- Cada pedido recibe un folio único y guarda el precio unitario que tenía cada artículo al confirmarse.
- La clave de idempotencia evita duplicar pedidos cuando se reintenta una solicitud.
- Los estados siguen el recorrido RECIBIDO → EN_PREPARACION → LISTO → ENTREGADO, con cancelación según las reglas del servidor.
- El cobro se registra al marcar la entrega, con el método EFECTIVO_AL_RECOGER.
- Sólo el dueño puede editar su puesto, menú y pedidos. Un cliente no puede usar las rutas del vendedor.
- Las cuentas, puestos y pedidos de demostración se cargan sólo si SEED_DEMO_DATA=true; el arranque normal no crea datos ficticios. Las categorías de referencia sí están disponibles.

4. Arquitectura y tecnologías

Componente	Implementación	Función
Aplicación Android	Kotlin, Jetpack Compose, Material 3, Navigation y ViewModels	Interfaz de cliente y vendedor, estado de pantallas y navegación.
Cliente de red	OkHttp, JSON con kotlinx.serialization	Solicitudes HTTP reales a la API Ktor, incluidas rutas PATCH.
API	Kotlin y Ktor 3	Autenticación, catálogo, pedidos, favoritos, calificaciones y administración del vendedor.
Persistencia	PostgreSQL, HikariCP y Flyway	Datos relacionales, conexiones, migraciones y transacciones.
Prueba local sin PostgreSQL	H2 en modo de compatibilidad PostgreSQL	Permite desarrollar en una laptop sin Docker; se desactiva en el despliegue.

El APK se comunica con la API mediante una URL configurable en compilación (API_BASE_URL). El APK entregable apunta a https://adelitagoat-api.onrender.com. La API se configura mediante DB_URL, DB_USER, DB_PASSWORD, DB_ALLOW_EMBEDDED_FALLBACK, SEED_DEMO_DATA y PORT.

5. Modelo de datos

La migración inicial define users, sessions, categories, stalls, products, favorites, orders, order_items y ratings. Las relaciones principales son: un vendedor posee puestos; cada puesto tiene productos y recibe pedidos; un pedido pertenece a un cliente y contiene líneas con precio histórico; cada calificación corresponde a un pedido entregado. Los importes se almacenan como centavos enteros para evitar errores de redondeo.

Flyway registra la versión de esquema y valida su checksum al reiniciar. La migración V1__init_schema.sql se conserva intacta para que una base ya inicializada pueda arrancar. Los cambios futuros de estructura deben añadirse como migraciones nuevas.

6. Contrato funcional de la API

Área	Rutas principales
Salud	GET /api/health
Acceso	POST /api/auth/register, POST /api/auth/login, POST /api/auth/logout, GET /api/auth/me
Catálogo	GET /api/categories, GET /api/stalls, GET /api/stalls/{id}, GET /api/products/{id}
Apartados	POST /api/orders, GET /api/orders, GET /api/orders/{id}
Favoritos y evaluación	Rutas de favoritos y calificación de pedidos entregados
Vendedor	GET /api/vendor/stalls, POST /api/vendor/stalls, PUT /api/vendor/stalls/{id}, gestión de productos y estados de pedidos

El detalle de métodos, cuerpos JSON y respuestas de error está en http-contract.md. Las rutas protegidas reciben un token de sesión. La API responde con errores de validación, autenticación, autorización, inexistencia o conflicto según corresponda.

7. Datos reales y datos de prueba

Los PDF y el diseño describen el caso de uso y muestran **ejemplos** de puestos, platillos y precios. No confirman el nombre, menú, precios, horario, ubicación puntual o contacto público de un comercio real. La auditoría de fuentes identifica estos datos pendientes. Por eso la base normal inicia sin comercios ni cuentas inventados y permite capturarlos desde la app de vendedor.

Los correos de prueba usan el dominio reservado example.invalid y sólo aparecen cuando se activa explícitamente el entorno demo. No se presenta un correo con dominio institucional como si fuera una cuenta autorizada.

8. Pruebas y evidencia

En la última verificación local se ejecutaron 10 pruebas automatizadas del servidor y 9 de Android, sin fallos, además de compilar el APK. Los casos incluyen base nueva sin datos demo, registro y acceso, creación y edición de puesto por un vendedor, rechazo de edición ajena, catálogo, precios e idempotencia de pedidos, estados, favoritos y métodos HTTP reales del cliente Android. El APK público de demostración se encuentra en output/apk/AdelitaGOAT-public-api-debug.apk y pesa 17 814 356 bytes. La configuración de la API no aparece en el inicio de sesión ni en el perfil; la URL se establece al compilar.

La revisión visual en emulador confirmó el arranque, la pantalla de acceso sin botones demo, el inicio de sesión de un vendedor nuevo y la aparición del formulario **Registrar puesto**. Estas observaciones se realizaron con una base local aislada de prueba; no representan un despliegue público ni un comercio real.

El 10 de noviembre se verificó la API publicada: /api/health respondió UP con database=CONNECTED y databaseMode=POSTGRESQL_SUPABASE_OR_REMOTE. Una prueba de extremo a extremo sobre HTTPS registró un vendedor y un cliente temporales, creó un puesto y un producto, y generó un pedido de dos unidades por 5 000 centavos. Después se eliminaron los registros de prueba; se confirmó que la base quedó con cero usuarios, puestos, productos y pedidos, y sólo seis categorías de referencia.

9. Ejecución y despliegue

Para desarrollo local, abrir android/ en Android Studio y ejecutar server/ con JDK 21. README.md contiene comandos y variables. El código está en GitHub, PostgreSQL está en el proyecto Supabase adelitagoat-escolar y la API Ktor se ejecuta en Render. El backend usa el Session Pooler de Supabase con TLS, usuario exclusivo y esquema privado adelita. El contenedor desactiva la base local y las semillas demo. Las credenciales se guardan como variables privadas de Render y no se incluyen en el repositorio ni en el APK.

Las tablas del esquema adelita no tienen RLS activado. Supabase señala esto en su inventario general; el esquema es privado y los roles anon y authenticated no tienen permiso USAGE sobre él. Si en el futuro se expone ese esquema mediante la Data API o se conceden permisos directos a otros roles, se deben activar RLS y políticas específicas antes de abrir el acceso.

El APK se compiló con API_BASE_URL=https://adelitagoat-api.onrender.com y ENABLE_DEMO_ACCOUNTS=false. Es una compilación **debug firmada para instalación y evaluación escolar**, no un paquete de publicación para Google Play. Queda pendiente validar el recorrido en un teléfono físico con conexión móvil y cargar el comercio real cuando se reciban sus datos autorizados.

10. Limitaciones y siguientes datos necesarios

El MVP no procesa pagos digitales ni envía notificaciones push. Los pedidos se consultan desde la app. Para cargar el comercio real faltan nombre autorizado, menú, precios, horario, ubicación y contacto público opcional. La infraestructura gratuita puede suspender servicios inactivos; verificar el arranque antes de la exposición.